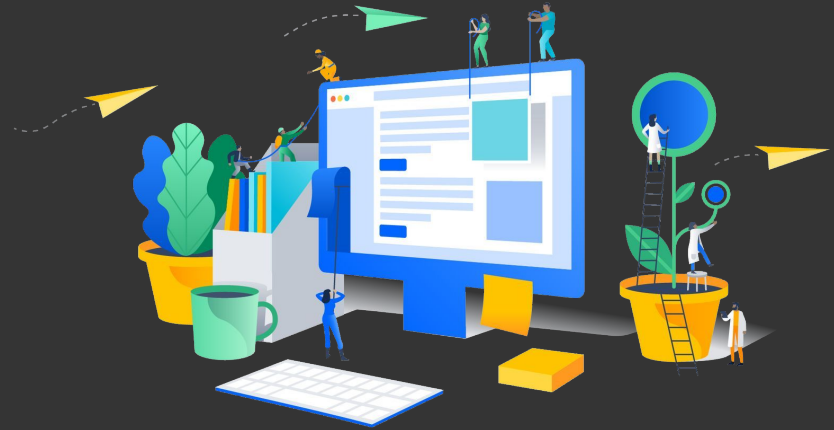
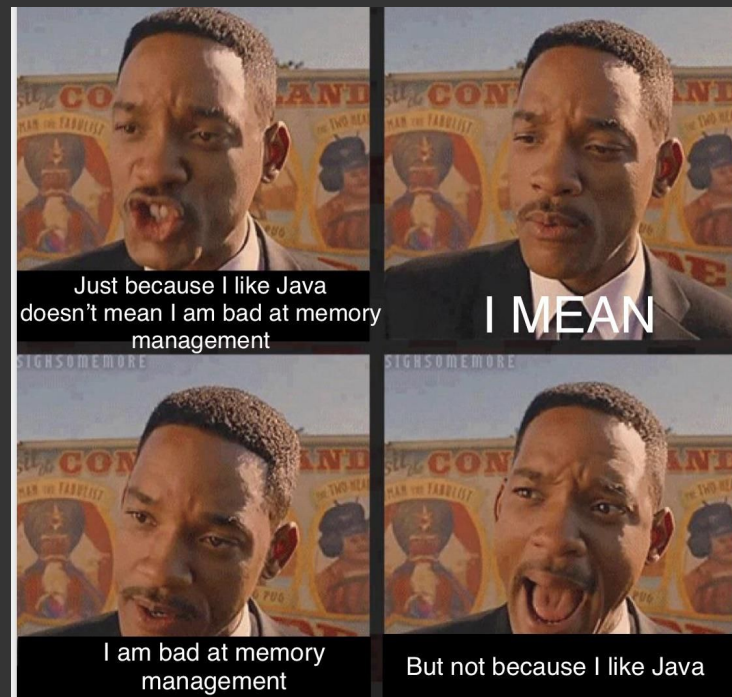


# Object Oriented Analysis and Design



# Memory Management

- There are languages that provide manual memory management (C, C++)
- There are languages that provide automatic memory management (e.g. Java)
- Pros and Cons to Both



# Garbage Collection

| GC Type            | Used in                | Strengths                          | Weaknesses                      |
|--------------------|------------------------|------------------------------------|---------------------------------|
| Reference Counting | Python, Swift, PHP     | Low overhead, immediate cleanup    | Fails with cyclic references    |
| Mark-and-Sweep     | Java, C#, JS           | Simple & widely used               | Can cause Stop-the-World pauses |
| Generational GC    | Java, .NET             | Optimized for short-lived objects  | Complex implementation          |
| Copying GC         | Lisp, Java (early)     | Efficient for young objects        | Wastes memory                   |
| Concurrent GC      | Java (ZGC, G1), Go     | Low latency, real-time performance | Higher CPU usage                |
| Incremental GC     | JavaScript, Lua        | Reduces lag spikes                 | Requires fine-tuning            |
| Tracing GC         | Haskell, Prolog        | Great for functional programming   | Slow for large graphs           |
| Region-Based GC    | Rust (Ownership Model) | Super-fast deallocation            | Not flexible                    |



# Smart pointers

- Smart pointers free us from having to call delete on new allocations
  - Prevents Memory leaks
  - Helps to handle dangling pointers
  - Reduces double calls to delete



# Unique pointer

- Allows only one unique reference to the original object
- Disable copy constructor
- Disable equal operator
- Allows only move semantic reference to be made



# Shared pointer

- Allow multiple source of references
- Implemented with ref counter
- Enable duplication and assignment



# Weak pointer

- Allow multiple source of references
- Implemented with two ref counter
- Wraps the shared pointer
- Let us to know if the original object no longer exists (Cache)
- Solves circular dependency problem  
(Manager <----> Worker)

